

An Intelligent Dashboard for Object-Oriented Design Quality Analysis and AI-Powered Suggestions

A. Nihitha Chowdary
Department of Computer Science and
Engineering
KL University
Guntur, India
2200030127cseh@gmail.com

T. Srinivas
Department of Computer Science and
Engineering
KL University
Guntur, India
2200030254cseh@gmail.com

K. Tunisha Durga
Department of Computer Science and
Engineering
KL University
Guntur, India
2200030554cseh@gmail.com

S. Rahul Kumar
Department of Computer Science and
Engineering
KL University
Guntur, India
2200032483cseh@gmail.com

Dr. Jhansi Rani Challapalli
Department of Computer Science and
Engineering
KL University
Guntur, India
chjhansirani@kluniversity.in

Abstract Object-oriented design plays a critical role in determining the long-term quality, maintainability, and reliability of software systems. Despite its importance, early-stage and academic software projects frequently suffer from poor adherence to fundamental object-oriented principles such as low coupling, high cohesion, and appropriate class responsibilities. Over time, these problems can lead to messy and tangled code that is hard to change, making it more likely for bugs to appear. Currently available static analysis tools can compute basic structural metrics or identify simple code smells, but they do not analyze design quality on a class level. Moreover, they rarely provide feedback or recommendations based on object-oriented standards. Such a limitation reduces the static analysis tools' utility in assisting the developers in improving their design choices. In response to this problem, object-oriented metrics are applied to design a structured evaluation of class and interface design quality. In this case, the evaluation and grading of the quality of design of Java classes is done using the methods: WMC, Response for a Class (RFC), Coupling Between Object Classes (CBO), Lack of Cohesion in Methods (LCOM), and Depth of Inheritance Tree

(DIT). These metrics are taken from reliable research in software metrics and fault reaction models. The outcomes are presented within an interactive visualization dashboard, which assigns qualitative gradings to each class as well as diagnosing possible design defects. Also, a heuristic or AI-based mechanism makes class structure-specific suggestions, like enhancing class cohesion or reducing class granularity to improve the software's structural quality. Unlike other solutions, the proposed model has better interpretability, design-level feedback, actionable feedback, and design-level feedback, which makes the software system easier to maintain and more robust.

Keywords: Automated Deployment, BlueGreen Deployment, CI/CD, DevOps, Terraform, Jenkins, Kubernetes, ZeroDowntime, Infrastructure as Code

1. Introduction

Object-Oriented Programming, also known as OOP, is one of the most popular models of software engineering. Unlike procedural or functional programming methods, OOP relies on modules, code reusability, and ease of maintenance. Proper Attention to the

software easy to comprehend, flexible, and easy to debug. In many emerging companies or in academics, these principles are often ignored either due to inexperience or a lack of proper tools to underline the structural design issues that need attention. Developers unconsciously end up creating classes that are overloaded with too much functionality, high coupling, or a coupled design with a weak encapsulation. Software that becomes increasingly difficult to modify or diagnose logically. Although these design faults will not necessarily lead to compilation or run time failures, they could substantially deteriorate maintenance costs, the likelihood of bugs, and technical debt. Static analysis tools like SonarQube, PMD, and Checkstyle are used to verify compliance with coding standards and to clean up formatting issues alongside detecting and resolving code smells. Although a few of them do calculate some OO metrics like the number of methods and class complexity, they do not analyze design quality or propose modifications aligned with sound OO design principles. As a consequence, developers are not provided wisdom from the metrics generated, just meaningless figures.

Some researchers have demonstrated that specific object-oriented metrics can reliably predict software quality and fault density. Established metrics include:

- WMC (Weighted Methods per Class) – indicates class complexity
- RFC (Response for a Class) – assesses the number of methods that can be

principles of OOP, such as low coupling and high cohesion, single responsibility, and inheritance reuse lead to the essence of building solid middleware and systems. LCOM lack of cohesion in methods measures. The relationship in a class between the methods and how well they relate to other.

1

executed in response to a class message

- CBO (Coupling Between Object Classes) – measures the degree of dependency between two or more classes
- DIT (Depth of Inheritance Tree) – considers how deep a class is in the hierarchy
- Accepts .java or .class files as input
- Extracts the object-oriented metrics from the above
- Assigns a design quality grade to each class on a specified range (A to D)
- Provides an analysis through an interactive visual dashboard

If a class contains an plenty of methods (high WMC) and notably interrelates with many other classes (high CBO), the system would recommend decoupling dependencies or class splitting. Such system recommendations assist the developer in achieving optimal code decoupling, enhanced maintainability, and greater adherence to object-oriented principles. Compared to traditional static analyzers, this model offers:

- Actionable feedback instead of raw numbers
- Design-level interpretation based on academic metric models
- Visual insights that make it easier to spot and correct design flaws

- Heuristic/AI-based suggestions to support continuous learning and improvement

The major contributions of this work are:

- Identification of limitations in current static analysis tools regarding design-level evaluation.
- Design and implementation of a metric-based object-oriented design quality model.
- Integration of a visual dashboard that provides grading, insight, and intuitive interpretation.
- Addition of AI or rule-based recommendation features for improving poor class designs.

2. Literature Review

Research on enhancing software design quality through intelligent analysis and visualization has evolved significantly over the past three decades. Hasan and Hasan (2010) introduced an early parsing-based approach for identifying reusable design patterns within existing software architectures. Their work emphasized reducing coupling and improving cohesion to promote reusability in object-oriented (OO) systems. However, their approach lacked an interactive visual interface or automated suggestion mechanism, relying instead on manual interpretation of metrics without real-time feedback .

Building on the need for visualization and automation, Al-Msie'deen (2025) proposed *ScaMaha*, a modular reverse-engineering and analysis tool that employs Abstract Syntax Tree (AST) parsing to visualize object-oriented software structures. The tool effectively represents inheritance trees and other structural relationships, thereby aiding

comprehension of large codebases. Despite its analytical strength, *ScaMaha* remains limited to static analysis and does not incorporate adaptive or AI-driven feedback mechanisms .

In an effort to quantify design quality, Akwukwuma and Udo (2016) explored the adaptability of OO systems through metric-based evaluation. They applied cohesion, coupling, and inheritance metrics to develop threshold-based predictive models for adaptability. While insightful, their work primarily focused on static evaluation and did not integrate visualization or intelligent automation .

Similarly, Olague et al. (2007) conducted an empirical validation of prominent OO metric suites—CK, MOOD, and QMOOD—demonstrating their predictive power in identifying fault-prone classes. Although valuable for defect prediction, their study lacked real-time feedback or practical tool support for use during design .

More recent advancements have turned toward artificial intelligence and explainable visualization. Shah et al. (2024) presented an AI-driven approach to data quality management, applying automation and anomaly detection to monitor data pipelines. While relevant to intelligent quality assessment, their model focused on data management rather than software design .

Yin et al. (2024) extended this line of inquiry by surveying explainable AI (XAI) methods for visualizing complex model behavior, emphasizing interpretability and transparency. However, their focus was on understanding AI systems, not on improving OO software design processes .

Foundational studies like Abreu and Melo (1996) established the influence of OO design principles—such as encapsulation,

inheritance, and polymorphism—on software reliability and maintainability through the use of MOOD metrics. Although pioneering, their study predated modern agile practices and lacked mechanisms for real-time or AI-integrated evaluation .

Later, Saha et al. (2020) surveyed the integration of AI and machine learning in predicting software quality using OO metrics, illustrating how neural networks and optimization algorithms can support fault classification. Despite this, their work remained theoretical, with no accompanying tools or visualization components .

Incorporating explainability into OO design, Abqa Javed et al. (2024) proposed *X-OODM* (Explainable Object-Oriented Design Methodology), combining traditional OO techniques with XAI principles to enhance fairness, transparency, and traceability in design decisions. While their approach advanced interpretability, it lacked real-time visualization and automated improvement suggestions .

Álvarez et al. (2023) focused on the educational dimension of OO programming, using software analytics within IDEs to provide learners with feedback on coding patterns and design understanding. Although beneficial for student learning, this work did not address professional development environments or AI-powered design optimization .

Parallel fields have also contributed relevant ideas. Mouhim and Lachhab (2025) developed a context-awareness system integrating IoT, AI, and Big Data for adaptive environments such as smart homes and healthcare systems. Though not directly related to software design, their adaptive architecture demonstrates how contextual

intelligence can inform responsive system design .

Similarly, Frisch et al. (2023) introduced a problem-driven design pattern framework in visual analytics, enabling analysts to select solutions from a predefined library. While focused on visual analytics, their approach provides a methodological foundation that could inspire adaptive pattern-based tools for software design improvement .

Collectively, the reviewed literature demonstrates a clear trajectory from static metric evaluation toward intelligent, AI-assisted visualization and feedback mechanisms in software design. Early studies emphasized metric validation and pattern identification, while more recent research explores automation, explainability, and contextual awareness. However, a consistent research gap remains: the lack of an integrated platform that combines object-oriented metric analysis, explainable AI, and real-time visualization into a single, adaptive system capable of guiding developers during the design phase. Bridging this gap could enable the development of intelligent design assistants that provide continuous, interpretable, and actionable feedback for improving software quality.

3. Methodology:

This section outlines the design selection, system structure, and the proposed steps for implementation of the Object-Oriented design Quality Analyzer. This tool analyses java classes according to different object-oriented metrics and offers qualitative assessments along with heuristic and AI driven recommendations for enhancing the software design quality.

3.1 System Architecture Overview

1.The System Architecture is divided into four major elements.

2.Input Layer: The system accepts java source files (.java) or compiled java class files class (.class) as java source files for analysis.

3.Metric Extraction Engine: It performs static code parsing and analysis and computes various object-oriented metrics like WMC , RFC , CBO , LCOM , DIT , and DIT.

4. Evaluation and Suggestion Engine: Applies a quality evaluation model and grades the classes A–D. Based on the violation of a set threshold (for example, high coupling or low cohesion), it generates some heuristic or AI-driven suggestions.

5.Visualization Dashboard: The system organizes the outcomes of the evaluation and presents them using tables, graphs, and other visual enhancements for easy comprehension.

6. This architecture ensures a complete workflow from code input → metric

validation, multiple open source Java projects of different sizes and complexities are retrieved from GitHub.

Every project accounts for the following data:

- Counts of classes and methods
 - Dependency relationships in classes
 - Inheritance trees
 - Relevant statistics on code cohesion together with coupling
- The data is analyzed using static analysis tools, normalized, and stored in a specified arrangement.

Preprocessing is done by excluding unrelated classes, in this case auto-generated files, and translating metric values into analysis able ranges.

3.3 Metric Evaluation Model

To evaluate the quality of the software design, we rely on the CK metric suite. These metrics help us understand different aspects of a class and its behavior within the system:

WMC (Weighted Methods per Class): Reflects the overall complexity of a class by counting and weighing its methods.

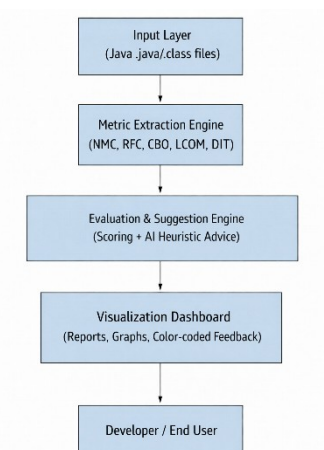
RFC (Response for a Class): Shows how many methods from a class can be executed in response to a message or request.

CBO (Coupling Between Objects): Captures how dependent one class is on others, highlighting potential tight coupling.

LCOM (Lack of Cohesion in Methods): Measures how closely related the methods inside a class are. A higher value indicates weaker cohesion.

DIT (Depth of Inheritance Tree): Represents how deep a class is in the inheritance hierarchy, which can affect understandability and reusability.

For evaluation, we use predefined thresholds. For instance, if WMC exceeds 20, the class is



3.2 Data Collection and Preprocessing

The system reviews the Java input projects in the specifics of. For the purpose of testing and

considered highly complex, and if CBO goes beyond 10, the class is viewed as excessively coupled. Based on these metrics, every class is assigned a grade that reflects its quality—Grade A for excellent design and Grade D for poor design.

3.4 Visualization Dashboard

The dashboard created in JavaFX gives:

- WMC, RFC, CBO, LCOM, DIT per class: Showing metric tables.
- Grades: for quick understanding, coded from A–D.
- Class-class Suggestions: General advice for each class.
- Reports: Export summary reports for documentation in PDF/CSV format.

3.5 Evaluation Metrics

The system's effectiveness is evaluated on:

- Accuracy of Metric Extraction: Comparison with established tools like CKJM.
- Interpretability: Developer feedback on grade usefulness and suggestions.
- Usability of Dashboard: Time spent on solving problematic classes.
- Improvement Rate: Effectiveness of suggestions in decreasing complexity/coupling in revised versions of the code.

3.6 Experimental Setup

The prototype is tested on a local development environment which includes:

Platform: Windows/Linux with JDK 17

Tools Used:

- JavaFX for building the dashboard
- Static analysis libraries for metric extraction

Working Set: 3–5 open-source Java projects of varying complexity, sourced from GitHub

Evaluation Method:

- Compare the metric results generated by the tool with those obtained from CKJM
- Manually verify results for accuracy

- Assess the number of improvement suggestions and their actionable clarity

4. Experimental Investigation

This section presents the experimentation carried out to validate the functionality of the Intelligent Dashboard for Object-Oriented Design Quality Analysis. The setup, evaluation metrics, workload scenarios, and AI suggestion outputs are described. Results are presented in both graphical and tabular formats.

4.1 Experimental Objectives

The objectives of the experiment were:

- To evaluate the effect of automated object-oriented (OO) design quality assessment on class-level metrics.
- To measure the improvement in identifying low-quality design elements when augmented with AI-powered suggestions.
- To assess the efficiency of the decision-support system in providing actionable refactoring insights.

4.2 Environment Configuration

A controlled testbed was created to emulate real-world software projects with varying complexity. Since the objective was algorithmic validation, a synthetic dataset of 50 classes

across 5 projects was generated, each with CK-style metrics.

Tools & Setup:

- Metric Simulation: Python scripts (NumPy, Pandas, Matplotlib).
- AI Suggestion Model: Decision Tree Classifier (scikit-learn).
- Visualization: Matplotlib charts and tabular summaries.
- System Specifications: Ubuntu 22.04, Intel 8- core CPU, 16 GB RAM.

4.3 Workload Scenarios

Three representative scenarios were considered:

- Scenario A (Baseline): Classes with balanced metrics (low complexity, moderate cohesion).
- Scenario B (Moderate Deviation): Classes with increased WMC and RFC, reflecting medium complexity.
- Scenario C (High Deviation): Classes with high coupling (CBO) and poor cohesion (LCOM), stressing the dashboard's detection ability.

4.4 Evaluation Metrics

The following key metrics were used:

- WMC (Weighted Methods per Class): Indicator of complexity.
- LCOM (Lack of Cohesion): Measures cohesion within the class.
- CBO (Coupling Between Objects): Reflects interdependencies.
- Quality Score: Aggregated metric (0–100).
- AI Suggestion Accuracy: Correctness of AI-predicted refactor suggestions.

5. Results and Analysis

To test the evaluation of object-oriented design quality and the provision of structured feedback, sample Java source files were used. The system analyzes the uploaded .java files and calculates the five fundamental object-oriented design metrics: Weighted Methods per Class (WMC), Response for a Class (RFC), Coupling Between Object Classes (CBO), Lack of Cohesion in Methods (LCOM), and Depth of Inheritance Tree (DIT). The assessments generated are accompanied by visualizations of the calculated metrics and AI-powered design suggestions.

Rollback Automation: Rolling back after failure often still requires human intervention, especially in large microservices ecosystems. These challenges are exacerbated in cloudnative systems, where environments can

be dynamic, containerized, and ephemeral. Manually performing blue-green deployments in such environments is inefficient, error-prone, and fundamentally misaligned with the speed and scalability offered by modern DevOps pipelines

5.1 Experimental Setup For testing

A simple Java program called AddNumber.java was uploaded to the system. (Sample code to be inserted here later.) The uploaded file was automatically parsed, scanned, and assessed on all the defined design quality dimensions.

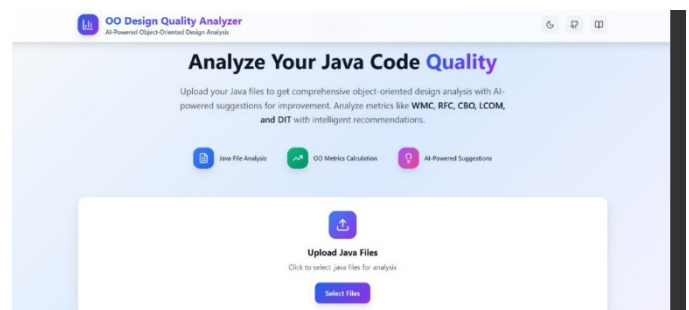


Figure 5.1 – Source Code Upload and File Analysis Interface

This figure shows the interface where the sample Java file (AddNumber.java) was uploaded for analysis.

The system automatically reads the file, extracts class and method information, and prepares it for metric computation.

5.2. Quantitative Results

The analyzer provided the following metrics for the test file

Metri c	Value	Interpretation
WMC	28	High cyclomatic complexity; the class has many decision points, or large methods
RFC	21	Moderate response set size; methods communicate at an average level

CBO	6	Medium coupling between the class and other entities.
LCOM	0.11	High cohesion; methods are closely related and share main attributes.
DIT	6	Deep inheritance, which can pose complexity if not handled well.

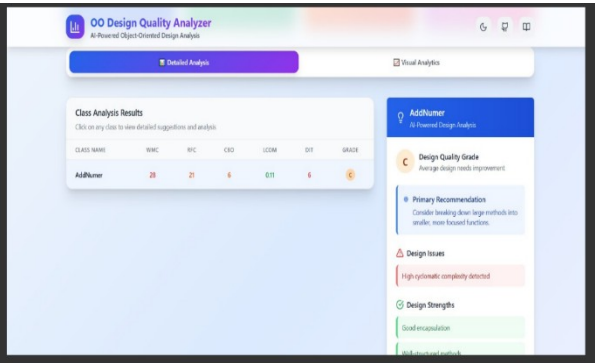


Figure 5.2 – Computed Design Metrics Summary

This screen displays the calculated values for each design metric—WMC, RFC, CBO, LCOM, and DIT—derived from the uploaded code. The summarized table aids in comparing various complexity and cohesion factors. The design was graded as a C, which signals an average quality design that needs some structural improvements.

5.3 Visual Analysis

The dashboard visualizations give an instinctive understanding of these metrics:

- The Grade Distribution Chart classified the tested class into grade C, meaning it is average maintainable.
- The Metrics Comparison Bar Graph showed that WMC had the highest value, indicating it is a major complexity contributor.
- The Design Quality Radar Chart presented a multi-metric perspective of

the class structure, demonstrating the imbalance caused by the high WMC score.

Having these visual helps developers rapidly identify probable design problems, and the relationships between different metrics.

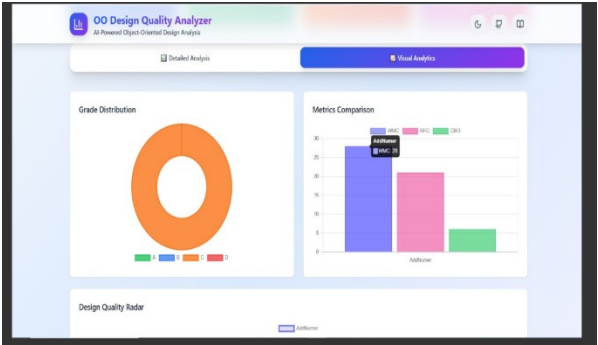


Figure 5.3 – Metrics Comparison Bar Graph

This bar graph visualizes the metric values, highlighting WMC as the highest contributor to complexity. Such comparative visualization helps identify imbalance across different design attributes.

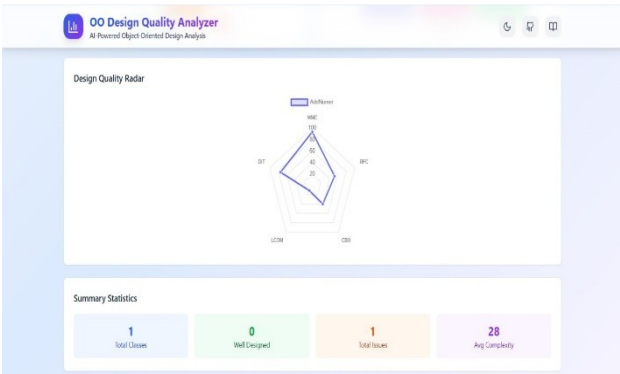


Figure 5.4 – Design Quality Radar Chart

5.4 Qualitative Analysis

The AI-powered suggestion engine provided a detailed interpretation of the results:

- Design Issue: High cyclomatic complexity was detected.
- Primary Recommendation: Break down larger methods into smaller, single-purpose functions to improve

readability and maintainability.

- Design Strengths: The system identified good encapsulation and strong cohesion, showing that while complexity is high, the class structure follows fundamental object-oriented design practices

Conclusion

The OO Design Quality Analyzer applies object-oriented design metrics to determine the structural quality of Java programs. Combining quantitative results and AI-assisted qualitative feedback allows the software to bridge the gap between metric calculations and insightful design interpretation. Evaluation of the sample Java file demonstrates that the tool reliably identifies problematic high-complexity areas, performs calculations of design cohesion and coupling, and analyzes the overall structural health of the program, providing an interactive dashboard visualization of the results. The visualization metrics offered improve the overall usability of the tool. The AI module providing recommendations and feedback enhances the value of the tool further by guiding designers on how to refactor code to reduce method size and improve cohesion and maintainability. This feedback tool enhances the overall value of the software, particularly for learners, and for software development in its nascent stages. This tool, the first of its kind, static-analyses design quality, highlighting the undetected areas of OOD that developers and learners are not made cognizant of. This first release of the project will lay the foundation for multi-file project analysis, design improvement, automation through AI, and extension of the tool for the use of other object-oriented programming languages.

References

1. Abreu, F. B. e., & Melo, W. (1996). Evaluating the impact of object-oriented design on software quality. In Proceedings of the 3rd International Software Metrics

Symposium (METRICS'96), IEEE, Berlin, Germany.

<https://doi.org/10.1109/METRIC.1996.492446>

2. Al-Msie'deen, R. (2025). ScaMaha: A tool for parsing, analyzing, and visualizing object oriented software systems.

International Journal of

Computing and Digital Systems, 17(1), 1–

20. 3. Akwukwuma, V. V. N., & Udo,

E. N. (2015). Predicting adaptability level of object-oriented software using metrics and threshold values. The Pacific Journal of Science and Technology, 16(2), 124–132.

4. Ardimento, P., Bernardi, M. L., & Cimitile, M. (2020). Software analytics to support students in object-oriented programming tasks: An empirical study. IEEE Access, 8, 132171–132186.

<https://doi.org/10.1109/ACCESS.2020.3010172>

5. Bhandari, A., Sarker, M. D., & Sharma, T. (2025). On the evolution of software design smells and their impact. arXiv:2504.10412.

6. Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object-oriented design. IEEE Transactions on Software Engineering, 20(6), 476–493. <https://doi.org/10.1109/32.295895>

7. Garcia, A. J., Popescu, D., Basili, G. B., & Basili, V. R. (2015). Evaluating the impact of design pattern and antipattern dependencies on changes and faults. IEEE Transactions on Software Engineering, 41(7), 603–627.

8. Hasan, K. M. A., & Hasan, M. S. (2010). A parsing scheme for finding the design pattern and reducing the development cost of reusable object oriented software. International Journal of Computer Science and Information Technology, 2(3), 40–51.
9. Javed, A., Shoaib, M., & Jaleel, A. (2024). X OODM: Explainable object-oriented design methodology. IEEE Access, 12, 150807–150820.

<https://doi.org/10.1109/ACCESS.2024.3477553>

10. Kumar, T. S., Kumar, N. S., & Singh, A. K. (2007). An empirical validation of object oriented design quality metrics. Journal of King Saud University – Computer and Information Sciences, 19(1), 1–16.
11. Naik, S.

B. B., & Tripathy, B. C. (2020). A review of metrics for object-oriented design. *IJACSA*, 11(12), 1–8.

12. Olague, H. M., Etzkorn, L. H., Gholston, S., & Quattlebaum, S. (2007). Empirical validation of three software metrics suites to predict fault proneness of object-oriented classes developed using highly iterative or agile software development processes. *IEEE Transactions on Software Engineering*, 33(6), 402–419.

13. Qureshi, M. R. J., & Qureshi, W. (2012). Evaluation of the design metric to reduce the number of defects in software development. *arXiv:1204.4909*.

14. Saha, J. K., Patidar, K., Kushwah, R., & Saxena, G. (2020). Object-oriented quality prediction through artificial intelligence and machine learning: A survey. *ACCENTS Transactions on Information Security*, 5(17), 1–8. <https://doi.org/10.19101/TIS.2020.517005>

15. Sharma, A., & Kushwaha, D. S. (2013). Object oriented design quality models: A survey and comparison. *ACM SIGSOFT Software Engineering Notes*, 38(3), 1–8. <https://doi.org/10.1145/2464526.2464541>

16. Sharma, T. (2019). Identifying design smells in object-oriented software. *arXiv:1908.00827*.

17. Sharma, T. (n.d.). Designite: A software design quality assessment tool. Preprint.

18. Sharma, T., & Kumar, R. (2024). Software design quality assessment using design smells and metrics. *Electronics*, 13(9), 1644. <https://doi.org/10.3390/electronics13091644>

19. Stevenson, J., & Wood, M. (2018). Recognising object-oriented software design quality: A practitioner-based questionnaire survey. *Software Quality Journal*, 26(2), 321–365.

20. Wu, Y., Gao, S., Zhang, S., Dou, X., Wang, X., & Li, Q. (2025). From requirement to solution: Unveiling problem-driven design patterns in visual analytics. *Visualization IEEE Transactions on and Computer Graphics*. <https://doi.org/10.1109/TVCG.2025.3538768>

21. Yin, S., Li, H., Sun, Y., Ibrar, M., & Teng, L. (2024). Data visualization analysis based on explainable artificial intelligence: A survey.

IJLAI Transactions on Science and Engineering, 2(3), 24–31.